

คู่มือระบบ Thaisausage ERP Integration

- Document version: 1.1.0
- Date: 16 September 2026
- Audience: ทีม IT / Operations, ทีม ERP และนักพัฒนาที่ดูแลระบบ
- Code baseline: working tree ณ วันที่ 16 กันยายน 2026 (commit ล่าสุด 7ce9101 บวกงาน DO callback ที่ยังไม่ commit)
- Production domain: `https://thaisausage.krs.co.th`

คู่มือนี้รวมข้อมูลจากการตรวจสอบโค้ดจริงทั้งหมดไว้ในเล่มเดียว ตั้งแต่การติดตั้ง การตั้งค่า API การ Deploy การดูแลประจำวัน ไปจนถึงผลการตรวจสอบ โปรเจคและข้อจำกัดที่ต้องแก้ก่อนใช้งานจริง หากข้อความในเอกสารอื่นขัดกับคู่มือนี้ ให้ยึดตามโค้ดและคู่มือนี้ (ดูหัวข้อ 14.3)

1. ระบบนี้คืออะไร

Thaisausage เป็น middleware ที่คั่นกลางระหว่าง ERP กับ eVRP ERP แจ้งว่ามี Sales Order (SO) พร้อมส่ง ระบบจะอ่าน SO ฉบับเต็ม ตรวจสอบถูกต้อง กั้นการส่งซ้ำด้วย SQLite แล้วส่งต่อไปยัง eVRP (`POST /v1/orders/import`) นอกจากนี้ระบบยังมีจุดรับ DO เข้าพื้นที่ staging เพื่อตรวจสอบ โดยจะไม่เขียนข้อมูลกลับ ERP

```
ERP SQL Server <-- SELECT-only approved_orders_query (scheduled, IsApprSo = 1) --> map --> validate --+
ERP JSON -----> /erp/orders -----> validate --+
ERP REST <-GET-- /erp/pull -----> map --> validate -----+
                                     v
                                     submission guard (SQLite submissions + order_claims) --> eVRP

eVRP DO -----> /vrp/do-received --> do_receipts (staged, never written back to ERP)
```

หลักการสำคัญ:

- ERP เป็นแหล่งข้อมูลหลักของ SO; ระบบใช้ Schedule และ query ที่กรองเฉพาะ `IsApprSo = 1`
- ระบบอ่าน SQL Server แบบ SELECT-only และไม่รับ SQL จาก HTTP request
- เมื่อ `dry_run=true` ระบบจะตรวจและคืน payload แต่ไม่ส่ง eVRP และไม่บันทึกการจอง request/order
- ระบบไม่ resend อัปเดตใหม่ถ้าผลลัพธ์ไม่แน่นอน รายการจะเป็น `needs_review` และต้องให้คนตรวจ

2. สถานะปัจจุบัน (16 ก.ย. 2026)

ความสามารถ	มีในโค้ด	พร้อมใช้งานจริง
รับ webhook SO (<code>/erp/hooks/order-ready</code>)	มี	legacy — บันทึก event ได้ แต่ไม่ใช่เส้นทางหลักและไม่ปลุก worker
ส่ง SO แบบ JSON ตรง (<code>/erp/orders</code>)	มี	dry-run เท่านั้น จนกว่าจะผ่าน eVRP UAT
ดึง SO จาก ERP REST (<code>/erp/pull</code>)	มี	ต้องตั้ง <code>erp.base_url</code> และ <code>ERP_TOKEN</code> ก่อน
SQL Server connector	มีโค้ด	ยังรอ query/mapping ที่อนุมัติจาก ERP
Sweep worker แบบ scheduled approved-SO	มี	ปิดอยู่ (<code>sync.enabled=false</code>)
รับ DO จาก eVRP (<code>/vrp/do-received</code>)	มี	ใช่ — staging เท่านั้น
DO staging เดิม (<code>/erp/do-received</code>)	มี	ใช่ — เก็บไว้เพื่อ compatibility
DO writer เข้า <code>tbl_D0hdr / tbl_Dodt1</code>	มีและต่อเข้ากับ callback แล้ว	ไม่ — ปิดอยู่ (<code>do_write.enabled=false</code>) และยังไม่มี mapping จาก DBA
ส่งข้อมูลจริงไป eVRP	มี connector	ยังไม่ผ่าน UAT
Swagger UI / OpenAPI	มี	ใช่
COD callback, เขียนกลับ ERP, retry worker, outbox, API ปลด/แก้ SO	ไม่มี	—

ความคืบหน้าตามแผน `process/features/erp-sqlserver/`: Phase 01 (Discovery) อยู่ในสถานะ TESTING เพราะยังขาด ODBC driver และ SO ตัวอย่างที่ได้รับอนุมัติ ส่วน Phase 02-06 ยังไม่เริ่ม

Production ที่ติดตั้งอยู่ตาม Operations Guide คือ VPS `krs-cloud` ใช้ Docker Compose + Caddy และตั้ง `dry_run=true`, `sync.enabled=false`

3. โครงสร้างโปรเจค

Path	หน้าที่
thaisausage/__main__.py	จุดเริ่ม โปรแกรม: โหลด config, สร้าง service, HTTP server และ thread sql-sweep
thaisausage/api.py	HTTP handler: auth, จำกัด body, routing, แปลง error เป็น HTTP status
thaisausage/service.py	business logic: submission guard, webhook inbox, DO staging, sweep และ SQLite schema
thaisausage/contracts.py	กติกา validate ของ standard SO JSON
thaisausage/connectors.py	HTTPS client ไป ERP REST และ eVRP รวมถึงฟังก์ชัน field mapping (mapped , lookup)
thaisausage/sqlserver.py	ODBC connection string, SELECT guard, fetch_orders และจัดกลุ่มแถวเป็น SO
thaisausage/do_writer.py	DO writer เข้า ERP: สร้าง INSERT แบบ parameterized, transaction เดี่ยว, ปิดเป็นค่าเริ่มต้น
thaisausage/config.py	อ่าน .env และ config JSON พร้อมตรวจสอบบังคับ
config/example.json	แม่แบบ config (สำเนาจริงคือ config/local.json ซึ่งถูก ignore)
config/do-write-template.json	แม่แบบ mapping ของ DO writer ครบ 55 + 24 คอลัมน์ตามลำดับของ ERP (ยังเว้น path ให้เติม)
examples/erp-order.json	ตัวอย่าง request สำหรับ /erp/orders
docs/openapi.json	OpenAPI 3.0 ที่ใช้แสดงผลใน /docs
deploy/	Dockerfile, docker-compose.yml, Caddyfile และ vps.env.example
tests/	unittest: integration (HTTP + mock upstream) และ SQL config
process/	แผนงาน, รายงาน phase และ context ของ workflow

ระบบใช้ Python 3.9 ขึ้นไปและ standard library เป็นหลัก ต้องใช้ pyodbc ร่วมกับ Microsoft ODBC Driver 18 เฉพาะเมื่อเชื่อม SQL Server (Docker image ติดตั้งให้แล้ว)

4. เริ่มใช้งานบนเครื่อง local

รันจาก root ของโปรเจค เพราะไฟล์ .env ถูกอ่านจาก working directory ปัจจุบัน:

```
cp config/example.json config/local.json
# สร้างไฟล์ .env อย่างน้อยต้องมี THAISAUUSAGE_API_KEY
python3 -m thaisausage --config config/local.json
```

เมื่อเริ่มสำเร็จจะเห็นข้อความ Thaisausage API http://0.0.0.0:8080 dry_run=True

```
curl http://127.0.0.1:8080/health
curl -X POST http://127.0.0.1:8080/api/v1/erp/orders \
  -H "Authorization: Bearer $THAISAUUSAGE_API_KEY" \
  -H 'Content-Type: application/json' \
  --data-binary @examples/erp-order.json
```

ระบบอ่าน config เฉพาะตอนเริ่ม ถ้าแก้ config/local.json หรือ .env ต้อง restart ทุกครั้ง

5. ตัวแปร environment (secrets)

Variable	ต้องใช้เมื่อ	หน้าที่
THAISAUUSAGE_API_KEY	ทุกกรณี (ถ้าไม่มี ระบบจะไม่เริ่ม)	Bearer key ที่ ERP ใช้เรียก API
VRP_TOKEN	dry_run=false (ถ้าไม่มี ระบบจะไม่เริ่ม)	ส่งไป eVRP ใน header X-Token
ERP_TOKEN	ใช้ /erp/pull	token สำหรับเรียก ERP REST
ERP_SQLSERVER_CONNECTION_STRING	sweep ทำงาน	ส่วน server/database/encryption ของ ODBC string
ERP_SQL_USER / ERP_SQL_PASSWORD	sqlserver.auth_mode=sql	credential แบบอ่านอย่างเดียว ระบบต่อท้ายให้ตอน connect

กติกาของ .env :

- ระบบรองรับรูปแบบ KEY=value บรรทัดที่ขึ้นต้นด้วย # เป็น comment และตัดเครื่องหมายคำพูดครอบคำออกให้
- ถ้า environment ของ shell หรือ container มีค่าอยู่แล้ว ค่านั้นมีสิทธิ์เหนือค่าในไฟล์
- .env และ deploy/vps.env อยู่ใน .gitignore ห้าม commit ห้ามใส่ในเอกสาร และห้ามถ่ายภาพหน้าจอที่มีค่าเหล่านี้
- ห้ามใส่ UID หรือ PWD ใน connection string เพราะระบบจะปฏิเสธ ให้ใช้ตัวแปรแยกแทน
- บน VPS ให้เก็บที่ /opt/thaisausage/.env และตั้ง permission เป็น 600

- `deploy/vps.env` ใช้เก็บข้อมูล SSH ของผู้ดูแล (`VPS_HOST` , `VPS_PORT` , `VPS_USER` , `VPS_SSH_KEY`) และไม่ได้ถูกโหลดเข้า container

6. การตั้งค่า `config/local.json`

ระบบใช้ไฟล์สองชั้นแยกกันเสมอ: `.env` เก็บ secret อย่างเดียว ส่วน `config/local.json` เก็บพฤติกรรมของระบบและห้ามมี secret เด็ดขาด (ใส่ได้แค่ *ชื่อ* ตัวแปร environment) ทั้งสองไฟล์อยู่ใน `.gitignore` และ `deploy/.dockerignore` ส่วน Docker image คัดลอกเฉพาะ `config/example.json` ขณะที่ `config/local.json` ถูก mount แบบ read-only ตอนรัน

บน VPS ต้องมีอย่างน้อย: `.env` → `THAISAUUSAGE_API_KEY` , `VRP_TOKEN` , `ERP_SQLSERVER_CONNECTION_STRING` , `ERP_SQL_USER` , `ERP_SQL_PASSWORD` และ `config/local.json` → `dry_run` , `sync.enabled` , `sync.interval_seconds` , `sqlserver.approved_orders_query` , `sqlserver.field_map` , `sqlserver.item_field_map` , `do_write.enabled` (ต้องเป็น `false`) พร้อม mapping ของ `do_write`

Key	ค่าเริ่มต้นใน example	ความหมาย
<code>host</code> , <code>port</code>	<code>0.0.0.0</code> , <code>8080</code>	address ที่ HTTP server รอรับ
<code>database</code>	<code>data/integration.sqlite3</code>	ไฟล์ SQLite (path สัมพันธ์กับ working directory)
<code>api_key_env</code>	<code>THAISAUUSAGE_API_KEY</code>	ชื่อตัวแปรที่เก็บ API key
<code>dry_run</code>	<code>true</code>	ต้องเป็น boolean เท่านั้น <code>false</code> คือส่งจริง
<code>sync.enabled</code>	<code>false</code>	เปิดหรือปิด scheduled worker ที่ตั้ง SO ที่อนุมัติแล้ว
<code>sync.interval_seconds</code>	<code>60</code>	รอบการทำงานของ scheduled worker
<code>vrp.base_url</code>	<code>https://vrp.oneplatformth.com/api</code>	ต้องเป็น HTTPS
<code>vrp.token_env</code> , <code>vrp.timeout_seconds</code>	<code>VRP_TOKEN</code> , <code>30</code>	token และ timeout ของ eVRP
<code>erp.base_url</code> , <code>orders_path</code>	ว่าง , <code>/api/orders</code>	ERP REST สำหรับ <code>/erp/pull</code>
<code>erp.token_env</code> , <code>auth_header</code> , <code>auth_prefix</code>	<code>ERP_TOKEN</code> , <code>Authorization</code> , <code>Bearer</code>	วิธีส่ง token ไป ERP
<code>erp.orders_list_path</code> , <code>items_path</code>	<code>data.orders</code> , <code>items</code>	ตำแหน่ง array ใน JSON ของ ERP
<code>erp.field_map</code> , <code>item_field_map</code>	map ชื่อเดียวกัน	key = field ปลายทาง, value = path ใน ERP
<code>sqlserver.connection_string_env</code> , <code>username_env</code> , <code>password_env</code>	ชื่อตามหัวข้อ 5	ชื่อตัวแปร secret
<code>sqlserver.auth_mode</code>	<code>sql</code>	<code>sql</code> หรือ <code>integrated</code> (Trusted_Connection)
<code>sqlserver.connect_timeout_seconds</code> , <code>query_timeout_seconds</code>	<code>10</code> , <code>30</code>	timeout ของ ODBC
<code>sqlserver.approved_orders_query</code>	ว่าง	SELECT ที่ผ่านการ review แล้ว ต้องมีคำว่า <code>IsApprSo</code> (เช่น <code>WHERE IsApprSo = 1</code>) มิฉะนั้นระบบปฏิเสธก่อนเชื่อมต่อ
<code>sqlserver.order_key</code>	<code>order_no</code>	ชื่อคอลัมน์ที่ใช้จัดกลุ่มแถวเป็น SO
<code>sqlserver.field_map</code> , <code>item_field_map</code>	ว่าง	map คอลัมน์ SQL ไปเป็น standard JSON
<code>do_write.enabled</code>	<code>false</code>	เปิดการเขียน DO เข้า ERP ห้ามเปิดจนกว่าจะผ่าน UAT และ sign-off
<code>do_write.header_table</code> , <code>detail_table</code>	ว่าง	ชื่อตารางปลายทางที่ผ่านการ review
<code>do_write.connection_string_env</code> , <code>username_env</code> , <code>password_env</code>	<code>ERP_SQLSERVER_WRITE_*</code> , <code>ERP_SQL_WRITE_*</code>	credential สำหรับเขียน ต้องคนละชุดกับบัญชี read-only
<code>do_write.header_columns</code> , <code>detail_columns</code>	ว่าง	mapping รายคอลัมน์ — คัดลอกโครงจาก <code>config/do-write-template.json</code> แล้วเติม path (ดู docs/do-insert-contract.md)
<code>do_write.detail_line_start</code>	<code>1</code>	เลขเริ่มต้นของ Slno

Key ต่อไปนี้อยู่ใน example แต่โค้ดยังไม่ได้อ่าน จึงยังไม่มีผลกับการทำงานจริง: `source.*` , `sqlserver.driver` , `sqlserver.encrypt` , `sqlserver.trust_server_certificate` , `sqlserver.approved_objects` , `sqlserver.approved_query_set` คำ `driver` และ `encryption` ที่ใช้จริงจะมาจาก `ERP_SQLSERVER_CONNECTION_STRING` เท่านั้น

Mapping รองรับ path ซ้อนกันด้วยจุด เช่น `"customer.code": "buyer.customer_code"` ถ้าไม่พบ path ระบบจะใส่ค่า `null` ระบบจะไม่แปลงชนิดข้อมูลภายใน ส่วนลด หรือหน่วยให้

7. API reference

7.1 การยืนยันตัวตนและข้อจำกัดทั่วไป

Endpoint สาธารณะที่ไม่ต้องใช้ key ได้แก่ `GET/HEAD /health` , `GET/HEAD /openapi.json` , `GET/HEAD /docs` และ `GET /favicon.ico` (ตอบ 204) ส่วน endpoint อื่นทั้งหมดต้องส่ง header ดังนี้:

Authorization: Bearer <THAISAUUSAGE_API_KEY>
Content-Type: application/json

ข้อจำกัดของ POST ทุกตัว:

- Content-Type ต้องเป็น application/json และต้องมี Content-Length ขนาด 1 ถึง 1,048,576 bytes
- ไม่รองรับ Transfer-Encoding (chunked) หากส่งมาจะได้ 422
- body ต้องเป็น JSON object และห้ามมี NaN หรือ Infinity
- connection มี timeout 35 วินาที
- ระบบไม่เขียน access log เพื่อไม่ให้ข้อมูลลูกค้าหรือ header รั่วไหล

7.2 สรุป endpoint

Method / Path	Auth	สำเร็จ	หน้าที่
GET /health	ไม่ต้อง	200	{status, dry_run} ตรวจสอบว่า process ทำงานอยู่ ไม่ได้ตรวจ SQL หรือ eVRP
GET /docs	ไม่ต้อง	200	Swagger UI (โหลดไฟล์จาก unpkg.com)
GET /openapi.json	ไม่ต้อง	200	OpenAPI contract
POST /api/v1/erp/hooks/order-ready	ต้อง	202	legacy — บันทึก event ลง erp_hooks เท่านั้น ไม่ไปลง scheduled worker และไม่ใช้เส้นทางหลัก
POST /api/v1/erp/orders	ต้อง	200/202	รับ standard SO JSON แล้ว validate และส่งต่อ
POST /api/v1/erp/pull	ต้อง	200/202	ดึง ERP REST หนึ่งครั้ง แล้ว map, validate และส่งต่อ
POST /api/v1/vrp/do-received	ต้อง	202	eVRP ส่ง DO เข้า staging
GET /api/v1/submissions/{request_id}	ต้อง	200/404	ผลที่บันทึกไว้ของการส่งจริง

7.3 Webhook: `POST /api/v1/erp/hooks/order-ready`

```
{
  "event_id": "ERP-S0-20260912-000001",
  "event_type": "sales_order.ready",
  "source_id": "main-erp",
  "company_id": "THAI",
  "order_no": "S0-0001",
  "changed_at": "2026-09-12T10:00:00+07:00"
}
```

Field	บังคับ	กติกา
event_id	ใช่	ไม่ว่างและยาวไม่เกิน 120 ตัว ควรใช้เฉพาะ A-Z a-z 0-9 . _ : - (ดู F5)
event_type	ใช่	sales_order.ready หรือ sales_order.changed
source_id	ใช่	รหัสแหล่ง ERP
order_no	ไม่บังคับใน API แต่ควรส่งทุกครั้ง	ใช้เป็นพารามิเตอร์ของ SQL query (ดู F4)
company_id, changed_at	ไม่	บันทึกไว้เพื่อใช้ตรวจสอบ ปัจจุบัน sweep ยังไม่ได้ใช้

ครั้งแรกระบบจะตอบ 202 {"event_id", "state": "received", "sweep": "queued"} ถ้าส่ง event_id เดิมพร้อม body เดิมซ้ำจะได้ 202 {"event_id", "state": "สถานะปัจจุบัน", "replayed": true} หาก body ต่างกันจะตอบ 409 และไม่ดำเนินการซ้ำ ทุกการแก้ไข SO จึงต้องใช้ event_id ใหม่

การตอบ 202 หมายความว่าบันทึก trigger แล้วเท่านั้น ไม่ได้แปลว่า SO ถึง eVRP แล้ว

7.4 ส่ง SO ตรง: `POST /api/v1/erp/orders`

ดูตัวอย่างเต็มได้ใน examples/erp-order.json กติกา validate มีดังนี้ (thaisausage/contracts.py):

Field	กติกา
request_id	1-120 ตัว ใช้ได้เฉพาะ A-Z a-z 0-9 . _ : -
orders	array จำนวน 1-100 รายการ
order_no	บังคับ ยาวไม่เกิน 100 และห้ามซ้ำใน batch
order_date	YYYY-MM-DD แบบ string
delivery_date	null, "" หรือ YYYY-MM-DD
payment_in_day	null หรือตัวเลขที่ ≥ 0 โดย 0 หมายถึง COD
pickup_hub_code	บังคับ
delivery_point_code / shipping_address	ต้องมียาวอย่างน้อยหนึ่งค่า
customer.code	บังคับ ยาวไม่เกิน 50
items	array จำนวน 1-2000 รายการ
items[].item_code	บังคับ ยาวไม่เกิน 50
items[].quantity	JSON number ที่ > 0
items[].unit_price	JSON number ที่ ≥ 0 (สินค้าแถมใช้ราคา 0 และแยกบรรทัด)
items[].cbm, nw	null หรือตัวเลขที่ ≥ 0

Field ที่ระบบยังไม่ validate ได้แก่ customer.name, description, pickup_hub_name และหน่วยสินค้า eVRP จะตรวจ master data อีกครั้งเมื่อส่งจริง ตัวเลขที่ส่งมาเป็น string เช่น "100" จะถูกปฏิเสธ

ผลลัพธ์:

- dry-run ตอบ 200 {state: "validated", dry_run: true, order_count, mapped_payload} โดยไม่บันทึกอะไรลงฐานข้อมูล
- live ตอบ 200 {state: "sent", vrp: {...}} เมื่อ eVRP ยืนยัน success: true หรือ 202 {state: "needs_review", reason} เมื่อผลไม่แน่นอน
- ส่ง request_id เดิมพร้อม payload เดิมจะได้ผลเดิมกลับมาพร้อม replayed: true
- request_id เดิมที่มี payload ต่างกัน หรือ order_no ที่เคยถูกจองภายใต้ request_id อื่น จะได้ 409 และระบบจะไม่ส่งทั้ง batch

7.5 ดึงจาก ERP REST: `POST /api/v1/erp/pull`

```
{"request_id": "ERP-PULL-0001", "query": {"page": 1}}
```

query ต้องเป็น object ที่มีแต่ค่า scalar ระบบจะแปลงเป็น query string แล้ว GET ไป erp.orders_path หนึ่งครั้ง ไม่มีการไล่น้ำอัดโนมิต ถ้าไม่มีรายการจะตอบ 200 {state: "empty"} ถ้ามีรายการระบบจะทำงานเหมือน /erp/orders ถ้าเรียก ERP ไม่สำเร็จจะได้ 502 ข้อควรระวังคือแม้อยู่ใน dry-run ระบบก็ยังเรียก ERP จริง

7.6 DO callback: `POST /api/v1/vrp/do-received`

eVRP เรียก endpoint นี้เมื่อสร้าง DO เสร็จแล้ว ระบบจะตรวจสอบ receipt_id / do_no, บันทึก payload ลง do_receipts และตอบ 202. การรับซ้ำด้วยข้อมูลเดิมเป็น replay ที่ปลอดภัย ส่วนข้อมูลเดิมแต่ payload เปลี่ยนจะตอบ 409.

การรับ DO ในระยะนี้เป็นการ staging เท่านั้น ไม่ส่งกลับ eVRP และไม่ Insert/Update/Delete ERP. ตัวเขียน tbl_D0hdr / tbl_Dodtl จะถูกเปิดได้เฉพาะหลังผ่าน UAT และได้รับอนุมัติแยกต่างหาก โดยใช้ do_write.enabled=true และ credential สำหรับเขียนคนละชุดกับบัญชีอ่าน SO.

ต้องมี receipt_id หรือ do_no (ยาวไม่เกิน 120) ระบบเก็บ payload เดิมพร้อม hash แล้วตอบ 202 รูปแบบของ field ภายใน data ยังรอผลสำรวจจาก ERP ตัวอย่าง request:

```
{"receipt_id": "VRP-D0-0001", "do_no": "D0-0001", "status": "completed", "data": {"so_no": "S0-0001", "delivered_at": "2026-09-16T12:00:00+07:00"}}
```

ตัวอย่าง response ครั้งแรกและตอนส่งซ้ำ:

```
{"receipt_id": "VRP-D0-0001", "receipt_key": "eVRP:VRP-D0-0001", "source": "eVRP", "state": "staged", "erp_write": false} {"receipt_id": "VRP-D0-0001", "receipt_key": "eVRP:VRP-D0-0001", "source": "eVRP", "state": "staged", "erp_write": false, "replayed": true}
```

เมื่อเปิด do_write.enabled=true และ payload มี transaction_no ระบบจะพยายามเขียน tbl_D0hdr / tbl_Dodtl ต่อทันทีหลัง staging แล้วรายงานผลในฟิลด์ do_write หลักการสำคัญคือ **staging ที่สำเร็จตอบ 202 เสมอ** ปัญหาของ writer ไม่เปลี่ยน HTTP status และ erp_write เป็น boolean ที่เป็น true เฉพาะตอน do_write.state = inserted เท่านั้น

do_write_state	ความหมาย	erp_write
inserted	เขียน Header/Detail สำเร็จและ commit แล้ว	true
disabled	do_write.enabled=false	false
skipped	ไม่มี transaction_no ใน payload จึงไม่เรียก writer	false
rejected	mapping/ข้อมูลไม่ผ่าน validate ไม่มี SQL ถูกส่ง	false
conflict	do_no หรือ transaction_no ถูกใช้โดย receipt อื่น	false
needs_review	ผลไม่แน่นอน เช่น timeout ระหว่าง commit	false

source เป็น eVRP เมื่อเข้าทาง /api/v1/vrp/do-received และเป็น erp เมื่อเข้าทาง endpoint เดิม (เส้นทาง ERP ไม่เรียก writer เลย) identity ที่ใช้จริงคือ receipt_key = "<source>:<receipt_id>" ดังนั้น receipt เลขเดียวกันจากสองช่องทางจะไม่ชนกัน การรับซ้ำของ identity ภายใน transaction เดียว (BEGIN IMMEDIATE) callback ที่มาพร้อมกันจึงได้ 202 พร้อม replayed: true ไม่ใช่ 500 eVRP ต้องใช้ receipt_id เดิมทุกครั้งที่ส่ง DO ในเดิมซ้ำ endpoint นี้ไม่เขียน ERP ไม่ว่ากรณีใด การเขียน DO เข้า ERP เป็นงานแยกที่ต้องเรียกภายในเท่านั้น (ดูหัวข้อ 9.1) และยังไม่มีย endpoint สาธารณะ

7.7 ตรวจสอบสถานะ: `GET /api/v1/submissions/{request\_id}`

endpoint นี้คืนผลที่บันทึกไว้ใน SQLite ซึ่งมีเฉพาะการส่งในโหมด live ดังนั้นในโหมด dry-run จะได้ 404 เสมอ

7.8 HTTP status

HTTP	ความหมาย	สิ่งที่ผู้เรียกควรทำ
200	validated / sent / empty / replay ของผลที่เสร็จแล้ว	บันทึกผล
202	บันทึก hook หรือ DO แล้ว หรือผลเป็น sending / needs_review	ห้ามถือว่าส่งถึง eVRP แล้ว
401	key ไม่มีหรือไม่ถูกต้อง	แก้ credential แล้วส่ง identity เดิมซ้ำ
404	path ไม่ถูกต้อง หรือไม่พบ submission	ตรวจ URL หรือ request_id
409	identity เดิมแต่ข้อมูลต่าง	หยุด automation แล้ว reconcile
422	JSON, header หรือ field ไม่ถูกต้อง	แก้ payload ถ้าข้อมูลเปลี่ยนให้ใช้ identity ใหม่
502	เรียก ERP REST ไม่สำเร็จ (/erp/pull)	retry ตามนโยบายของ operator
500	internal error	แจ้งผู้ดูแลระบบ

7.9 สถานะ submission และ hook

State	ใช้กับ	ความหมาย
received	hook	บันทึกแล้ว รอ sweep
validated	submission, hook	ผ่าน validate ใน dry-run โดยยังไม่ได้ส่งจริง
sending	submission, hook	จองแล้ว กำลังส่ง ถ้าค้างอยู่แสดงว่า process หยุดกลางทาง ต้องตรวจเอง
sent	submission, hook	eVRP ตอบ success: true
needs_review	submission, hook	ผลไม่แน่นอน: upstream_response_not_confirmed , upstream_http_NNN , upstream_outcome_unknown , connector_error
review	hook	sweep ไม่พบ SO หรือเกิด error (ระบบไม่ได้เก็บเหตุผลไว้ ดู F6)
staged	DO	เก็บเข้า staging แล้ว

8. การทำงานของ scheduled worker

thread sql-sweep เริ่มพร้อม HTTP server และทำงานเป็นรอบดังนี้:

- รอจนครบ sync.interval_seconds แล้วทำงานหนึ่งรอบ ไม่มีสิ่งใดมาปลุกก่อนกำหนด (hook ไม่ปลุก worker แล้ว)
- ถ้า sync.enabled=false ระบบข้ามรอบนั้นไป แต่ HTTP API ยังทำงานตามปกติ
- เรียก sqlserver.approved_orders_query ที่ผ่านการ review แบบ SELECT-only ไม่มีพารามิเตอร์ และไม่รับ SQL จาก HTTP
- query ต้องกรองเฉพาะ SO ที่อนุมัติแล้ว (IsApprSo = 1) ระบบเชื่อผลลัพธ์ของ query นี้ จึงไม่มีตัวกรองซ้ำในโค้ด
- แต่ละ SO ใช้ request_id = "SCHEDULE-" + sha256(order_no)[:32] แล้วผ่าน submission guard เดียวกับ endpoint อื่น
- SO ที่ส่งสำเร็จแล้วได้ skipped เหตุผล already_sent ; SO ที่ครั้งก่อนค้างที่ sending / needs_review จะได้ needs_review เหตุผล prior_attempt_* และระบบไม่ส่งซ้ำให้เอง

- SO ที่มีบรรทัดสินค้าไม่มี `item_code` จะได้ `review` เหตุผล `detail_item_code_missing` และไม่ถูกส่งทั้งใบ
- ถ้าอ่าน query ทั่วๆไปไม่ได้ (เช่นแถวไม่มี `order key`) รอบนั้นคืนผลเดียวเป็น `review` โดยไม่โยน exception ออกไป
- แต่ละ SO ถูกแยกจากกัน ถ้ารายการใดทั้งจะได้ `review` พร้อมเหตุผลและเขียน log แบบ redact ส่วนรายการที่เหลือยังส่งต่อไปได้ตามปกติ
- แถวที่ไม่มี `order_no` จะได้ `review` เหตุผล `order_no_missing` โดยไม่พยายามส่ง

พฤติกรรมที่ผู้ดูแลต้องรู้:

- เมื่อ `dry_run=true` ระบบจะ validate และ preview เท่านั้น ผลลัพธ์เป็น `preview` เหตุผล `dry_run_preview` ไม่มีการส่ง eVRP และไม่มีการจอง `order_no`
- SO ที่ส่งไปแล้วและถูกแก้ไขภายหลังจะชนการจอง `order_no` กลายเป็น `review` ต้องแก้ที่ eVRP ด้วยมือ เพราะยังไม่มี contract สำหรับ update หรือ cancel
- `erp_hooks` และ `sweep_hooks` ยังอยู่ในโค้ดเพื่อ compatibility แต่ไม่ได้ถูกเรียกจาก worker ปัจจุบัน

9. SQL Server connector

การประกอบ connection string: ระบบนำ `ERP_SQLSERVER_CONNECTION_STRING` มาต่อท้ายด้วย `UID={user};PWD={password}` (escape คำด้วยวงเล็บปีกกา) เมื่อใช้ `auth_mode=sql` หรือต่อท้าย `Trusted_Connection=yes` เมื่อใช้ `integrated`

```
DRIVER={ODBC Driver 18 for SQL Server};SERVER=erp-db.example,1433;DATABASE=ERP;Encrypt=yes;TrustServerCertificate=no;Connection Timeout=10
```

ก่อนเปิด connection ทุกครั้ง ระบบบังคับสัญญาของ query ดังนี้ (ตรวจด้วยข้อความ ไม่ใช่ SQL parser จึงเลือกปฏิเสธไว้ก่อนเมื่อไม่มั่นใจ):

- ต้องเป็นคำสั่งเดียว ห้ามมี ;
- ห้ามมี comment `--` หรือ `/* */` เพื่อไม่ให้ซ่อนเงื่อนไข
- ต้องขึ้นต้นด้วย `SELECT` และห้ามมีคำว่า `INSERT`, `UPDATE`, `DELETE`, `MERGE`, `EXEC`, `ALTER`, `DROP`, `CREATE`, `TRUNCATE`, `GRANT`, `REVOKE`, `DENY`, `BACKUP`, `RESTORE`, `WAITFOR`, `BULK`, `OPENROWSET`, `OPENQUERY`, `OPENDATASOURCE` หรือ `INTO`
- เฉพาะ `approved_orders_query` ต้องมีเงื่อนไข `IsApprSo = 1` ที่มองเห็นได้ และห้ามเทียบ `IsApprSo` กับค่าอื่น เช่น `= 0`, `<> 1` หรือ `>= 1`
- ถ้า view กรอง `approved` ไว้แล้ว ก็ยังต้องเขียน `IsApprSo = 1` ใน query เพื่อให้ตรวจสอบได้จากไฟล์ config

guard นี้ตรวจข้อความ ไม่ใช่กลไกความปลอดภัยหลัก บัญชี SQL จึงยังต้องมีสิทธิ์ `SELECT` เฉพาะ view ที่อนุมัติเท่านั้น (ดู F11)

การแปลงแถว SQL เป็น SO:

- แถวผลลัพธ์จะถูกจัดกลุ่มตามคอลัมน์ `order_key` โดยใช้แถวแรกของกลุ่ม map header ด้วย `field_map`
- ทุกแถวถูก map ด้วย `item_field_map`; ถ้าบรรทัดใดไม่มี `item_code` ที่ใช้ได้ ทั้ง SO จะถูกทำเครื่องหมาย `rejected_reason = detail_item_code_missing` และ scheduler จะไม่ส่งใบนั้น
- ชื่อคอลัมน์ต้องไม่มีจุด เพราะจุดใช้แบ่ง path ให้ตั้ง alias ใน SQL แทน
- connector จะแปลง `date / datetime` เป็น ISO string และ `Decimal` เป็นตัวเลข JSON ให้ตรง contract; ยังต้องยืนยันผลกับ SQL จริงแบบ read-only (ดู F3)

ตัวอย่างโครง config (ชื่อ view และคอลัมน์เป็นตัวอย่าง ต้องแทนด้วยผล Phase 01):

```
{
  "sqlserver": {
    "orders_query": "SELECT h.so_no AS order_no, CONVERT(char(10), h.so_date, 23) AS order_date, h.credit_days AS payment_in_day, h.cust_code AS customer_code, h.ship_addr AS shipping_address, h.hub_code AS pickup_hub_code, l.line_no AS line_no, l.item_code AS item_code, l.item_name AS description, CAST(l.qty AS float) AS quantity, CAST(l.price AS float) AS unit_price FROM dbo.v_so_ready h JOIN dbo.v_so_lines l ON l.so_no = h.so_no WHERE h.so_no = ?",
    "order_key": "order_no",
    "field_map": {"order_no": "order_no", "order_date": "order_date", "payment_in_day": "payment_in_day", "customer.code": "customer_code", "shipping_address": "shipping_address", "pickup_hub_code": "pickup_hub_code"},
    "item_field_map": {"line_no": "line_no", "item_code": "item_code", "description": "description", "quantity": "quantity", "unit_price": "unit_price"}
  }
}
```

ตรวจการเชื่อมต่อแบบอ่าน metadata อย่างเดียว (ไม่แตะตารางธุรกิจ):

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import json; from thaisausage.sqlserver import SQLServerConnector; c=json.load(open('/app/config/local.json')); print(SQLServerConnector(c['sqlserver']).check_read_only_connection())"
```

9.1 DO writer เข้า ERP (`do\_write`)

สถานะ:  CODE DONE — โครงสร้างพร้อมและมี test แต่ยังไม่มี mapping จริงและยังไม่เคยเขียน ERP รายละเอียด mapping ทั้งหมดอยู่ใน docs/do-insert-contract.md

กติกาลดความปลอดภัยที่บังคับในโค้ด:

- ค่าเริ่มต้น `do_write.enabled=false` เปิดได้จาก config เท่านั้น ห้ามเปิดจาก payload และยังไม่ endpoint สาธารณะ
- ต้องใช้ credential คนละชุดกับบัญชี read-only ถ้าตั้ง env เดียวกับ `ERP_SQL_USER / ERP_SQL_PASSWORD / ERP_SQLSERVER_CONNECTION_STRING` ระบบจะปฏิเสธ
- validate mapping และ payload ให้เสร็จก่อนเปิด connection เสมอ
- ชื่อตารางและคอลัมน์มาจาก config ที่ review แล้ว ค่าทุกค่าถูกผูกเป็น parameter ไม่มีการต่อ string จาก payload
- Header และ Detail อยู่ใน transaction เดียว ถ้าบรรทัดใดล้มเหลวจะ rollback ทั้งชุด
- `EntryDate` ใช้ `GETDATE()` ของ SQL Server ส่วน `IsAcc=0` , `IsAccBy` , `IsAccDate` , `DocuNw` เป็นค่าคงที่ตาม contract
- เมื่อ `dry_run=true` ระบบจะ validate และนับ statement เท่านั้น ไม่เปิด connection

สถานะที่บันทึกในตาราง `do_writes`:

State	ความหมาย	เขียนซ้ำได้
preview	dry-run ตรวจ mapping แล้ว ไม่ได้เขียน	ได้
disabled	flag ปิดอยู่	ได้
rejected	validate ไม่ผ่าน ไม่มี SQL ถูกส่ง	ได้ (หลังแก้สาเหตุ)
inserted	commit สำเร็จ	ไม่ได้
needs_review	ผลไม่แน่นอน เช่น timeout ระหว่าง commit	ไม่ได้ ต้องตรวจ ERP ด้วยมือก่อน

การกันซ้ำใช้ `receipt_id` เป็น primary key พร้อม unique index บน `do_no` และ `transaction_no` ถ้า payload ของ receipt เดิมเปลี่ยนหลังเคยเขียนแล้วจะได้ conflict ไม่ใช้การเขียนซ้ำ

10. ข้อมูลใน SQLite

```
submissions(request_id PK, payload_hash, state, result JSON, created_at)
order_claims(order_no PK, request_id)
erp_hooks(event_id PK, event_type, source_id, company_id, order_no, changed_at, state, received_at)
do_receipts(receipt_key PK, source, receipt_id, do_no, payload_hash, payload, state, received_at)
do_writes(receipt_key PK, source, receipt_id, do_no UNIQUE, transaction_no UNIQUE, payload_hash, state, reason, updated_at)
```

ใน container ไฟล์ฐานข้อมูลอยู่ที่ `/app/data/integration.sqlite3` บน volume `thaisausage-data` ตาราง `submissions` เก็บเฉพาะ hash, สถานะ และเลขอ้างอิงจาก eVRP ไม่ได้เก็บ payload เต็ม ดังนั้นจึงต้องเก็บไว้ที่ ERP เพื่อใช้ reconcile

ดูจำนวนรายการตามสถานะ:

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import sqlite3; db=sqlite3.connect('data/integration.sqlite3');
print('hooks', db.execute('SELECT state, count(*) FROM erp_hooks GROUP BY state').fetchall()); print('submissions', db.execute('SELECT
state, count(*) FROM submissions GROUP BY state').fetchall())"
```

สำรองข้อมูลโดยใช้ SQLite backup API ซึ่งทำได้ขณะระบบทำงาน:

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import sqlite3;
sqlite3.connect('data/integration.sqlite3').backup(sqlite3.connect('/tmp/backup.sqlite3'))"
docker compose -f deploy/docker-compose.yml cp thaisausage:/tmp/backup.sqlite3 ./integration-$(date +%F).sqlite3
```

ห้ามลบแถวใน `order_claims` เพื่อบังคับส่งซ้ำ และห้าม restore backup เก้าทัชขณะเปิดโหมด live ก่อน reconcile กับ eVRP

11. Deploy บน VPS

Service	รายละเอียด
thaisausage	Python 3.11 slim + ODBC Driver 18 รันด้วย user <code>appuser</code> (uid 10001) เปิดพอร์ต 8080 เฉพาะใน network ภายใน root filesystem เป็น read-only, <code>/tmp</code> เป็น tmpfs และมี healthcheck เรียก <code>/health</code> ทุก 30 วินาที
caddy	Caddy 2 รับพอร์ต 80/443 ออก HTTPS อัตโนมัติสำหรับ <code>thaisausage.krs.co.th</code> แล้ว reverse proxy ไปที่ app และเริ่มหลังจาก app healthy แล้ว
Volumes	<code>thaisausage-data</code> (SQLite), <code>caddy-data</code> , <code>caddy-config</code>
Mount	<code>config/local.json</code> แบบ read-only และ <code>.env</code> ผ่าน <code>env_file</code> (format raw)

ข้อกำหนดก่อน deploy:

- DNS A record ของ `thaisausage.krs.co.th` ต้องชี้มาที่ VPS และพอร์ต 80/443 ต้องเปิดเพื่อให้ออก certificate ได้
- image ถูก build สำหรับ amd64 เพราะ repo ของ Microsoft ระบุ `arch=amd64` ไว้ตายตัว

- build context คือ root ของ repo จึงต้องใช้ `.dockerignore` ที่ root เท่านั้น (ไฟล์ใน `deploy/` ไม่มีผล) ไฟล์นี้กัน `.env`, `config/local.json`, `deploy/vps.env`, `data/`, `tests/`, `process/` และ PDF ไม่ให้เข้า build context
- ต้องมี `/opt/thaisausage/.env` (permission 600) และ `config/local.json` บน VPS

Deploy ครั้งแรกหรืออัปเดต:

```
cd /opt/thaisausage
git fetch origin main
git reset --hard origin/main # ทิ้งการแก้ไขไฟล์ที่ track บน VPS; .env และ config/local.json ถูก ignore จึงไม่หาย
docker compose -f deploy/docker-compose.yml build
docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml ps
curl -fsS https://thaisausage.krs.co.th/health
```

ก่อนอัปเดตให้สำรอง SQLite (หัวข้อ 10) เสมอ หลังอัปเดตให้ตรวจ `/health`, `/docs`, log และลองส่ง webhook กับ DO แบบ mock ได้แต่ที่ยังไม่ commit และ push จะไม่ขึ้นไปบน VPS ดังนั้น ณ วันที่เขียนเอกสาร ระบบบน production ยังไม่มีการปลุก sweep ด้วย webhook, การรองรับ HEAD, favicon และการแก้ CSP ของ Swagger

12. การดูแลประจำวันและการแก้ปัญหา

12.1 รายการตรวจประจำวัน

- [] `docker compose ps` ต้องแสดงทั้งสอง service เป็น running และ app เป็น healthy
- [] `https://thaisausage.krs.co.th/health` ต้องตอบ 200 และค่า `dry_run` ต้องตรงกับที่อนุมัติไว้
- [] นับ hook ที่เป็น `received` ค้างนาน และ hook ที่เป็น `review`
- [] นับ submission ที่เป็น `sending` หรือ `needs_review`
- [] ตรวจพื้นที่ดิสก์ของ volume และตรวจว่ามี backup ล่าสุด
- [] ตรวจอายุ certificate ของ Caddy

12.2 อาการและวิธีแก้

อาการ	สาเหตุที่เป็นไปได้	วิธีแก้
ระบบไม่เริ่ม: <code>dry_run must explicitly be true or false</code>	ค่าใน config ไม่ใช่ boolean	แก้เป็น <code>true</code> หรือ <code>false</code> โดยไม่มีเครื่องหมายคำพูด
ระบบไม่เริ่ม: Set the environment variable named by <code>api_key_env</code>	ไม่มี <code>THAISAUUSAGE_API_KEY</code>	ตรวจ <code>.env</code> หรือ <code>env_file</code> แล้ว restart
ระบบไม่เริ่ม: Live mode requires the VRP token	<code>dry_run=false</code> แต่ไม่มี <code>VRP_TOKEN</code>	ใส่ token หรือกลับไปใช้ <code>dry-run</code>
ระบบไม่เริ่ม: <code>KeyError</code>	config ขาด key หลัก เช่น <code>vrp</code> หรือ <code>erp</code>	เทียบกับ <code>config/example.json</code>
401 unauthorized	key ผิด หรือขาดคำว่า <code>Bearer</code>	ตรวจ header
422 Content-Type must be application/json	client ส่ง form หรือ chunked	ตั้ง header และส่ง Content-Length
502 <code>erp_request_failed</code>	ERP REST ไม่ตอบ, ไม่ใช่ HTTPS หรือเจอ redirect	ตรวจ <code>erp.base_url</code> และ token (ระบบไม่ follow redirect)
hook ค้างที่ <code>received</code>	<code>sync.enabled=false</code> หรือ sweep error ทั้งรอบ	เปิด sync ตาม gate และตรวจ SQL config และ driver
hook เป็น <code>review</code>	ไม่พบ SO, SQL error, validate ไม่ผ่าน, <code>order_no</code> ซ้ำ, <code>order_no</code> หาย หรือข้อมูลเก่าที่ไม่ทราบ hash	ทดสอบ query ด้วย <code>order_no</code> นั้นแบบ read-only แล้วตรวจ reason/mapping
submission เป็น <code>needs_review</code>	eVRP timeout, error หรือตอบไม่ตรงรูปแบบ	ตรวจที่ eVRP ด้วยเลข SO ก่อน ห้ามส่ง payload ที่เปลี่ยนแล้วซ้ำ
<code>/docs</code> หน่วง	browser โหลด unpkg.com ไม่ได้	ใช้ <code>/openapi.json</code> กับ Postman หรือ Swagger Editor แทน
app unhealthy	process ค้าง หรือ config ผิด	<code>docker compose logs --tail=100 thaisausage</code>
HTTPS ใช้ไม่ได้	DNS ผิด หรือพอร์ต 80/443 ถูกปิด	ตรวจ DNS, firewall และ <code>docker compose logs caddy</code>

12.3 ขั้นตอนเปิดใช้งานจริง (ย่อ)

- Gate A: ติดตั้ง ODBC แล้ว, ผ่าน metadata check (หัวข้อ 9) และบัญชี SQL เป็น read-only จริง

- Gate B: ใส่ `approved_orders_query` และ mapping แล้ว ทดสอบกับ SO ที่อนุมัติใน dry-run ต้องได้ JSON ถูกต้องครบทุกกรณี (COD, credit, ของแถม, หลายบรรทัด, ภาษาไทย, ทศนิยม)
- Gate C: ทดสอบกับ eVRP UAT โดยใช้ token ทดสอบ แล้วตรวจเลขอ้างอิงและยอดเงิน
- Gate D: ซ้อม backup/restore, กำหนดผู้รับผิดชอบ แล้วตั้ง `dry_run=false` ก่อน จากนั้นจึงตั้ง `sync.enabled=true` และเฝ้าดูรอบแรก
- F1-F3 แก่แล้วและมี regression test; Gate B ยังต้องผ่านการเชื่อม SQL จริงแบบ read-only กับ SO ที่อนุมัติ รายละเอียดอยู่ใน Operations Guide และ Integration Test Plan

13. การทดสอบ

```
python3 -m unittest discover -s tests -v
```

ผลวันที่ 16 ก.ย. 2026: ผ่าน 98 จาก 98 test ครอบคลุม HTTP auth, validation, idempotency, การส่งซ้ำพร้อมกัน, timeout, DO staging/callback (replay, conflict, concurrent), scheduled sweep, SQL guard, credential ของ SQL และชุดทดสอบ DO writer (parameter binding, transaction, rollback, duplicate, dry-run, flag ปิด และการสร้าง statement ตามคอลัมน์จริงของ ERP) ทุก test ใช้ mock โดยไม่เรียก ERP หรือ eVRP จริง และไม่มี INSERT เข้า ERP เกิดขึ้น

ส่วนที่ยังไม่มี test เชื่อม SQL Server จริงคือ connection/query กับ schema ของ ERP; มี unit test สำหรับ `_group_rows` และชนิดข้อมูล `Decimal/date` แล้ว แต่ยังต้องทำ read-only verification กับ SO ที่อนุมัติ

14. ผลการตรวจสอบโปรเจค

14.1 ประเด็นที่ตรวจพบและสถานะแก้ไข

ID	ระดับ	ประเด็น	ตำแหน่ง	ผลกระทบ / หลักฐาน	ข้อเสนอ
F1	สูง	mapped ไม่ได้ import	thaisausage/sqlserver.py	แก้แล้ว: import โดยตรงและมี regression test <code>_group_rows</code>	
F2	สูง	sweep ใน dry-run ทำให้ hook ถูกใช้ไปแล้ว	service.py	แก้แล้ว: dry-run คง Hook เป็น <code>received</code> พร้อม reason <code>dry_run_preview</code>	
F3	สูง	ชนิดข้อมูลจาก pyodbc (Decimal, date) ไม่ผ่าน validate	sqlserver.py	แก้แล้ว: normalize เป็น JSON-compatible type และมี unit test; ยังต้องทดสอบ SQL จริงแบบ read-only	
F4	กลาง	hook ที่ไม่มี <code>order_no</code> ทำให้ query ไม่มีเงื่อนไข	service.py	แก้แล้ว: ตั้ง <code>review/order_no_required</code> และไม่เรียก SQL	
F5	กลาง	ชุดอักขระของ <code>event_id</code> ไม่ตรงกับ <code>request_id</code>	service.py	แก้แล้ว: validate ด้วย pattern เดียวกับ <code>request_id</code>	
F6	กลาง	sweep error ถูกกลืนเจียบๆ และไม่เก็บ reason	<code>__main__.py</code> , <code>service.py</code>	แก้แล้ว: log แบบไม่เปิดเผย payload และเก็บ reason ต่อ Hook	
F7	กลาง	hook ซ้ำไม่ได้เทียบ body	service.py	แก้แล้ว: เก็บ payload hash และตอบ 409 เมื่อ <code>event_id</code> เดิมมีข้อมูลต่างกัน	
F8	ต่ำ	config หลาย key ไม่มีผลกับการทำงาน	config/example.json	ทำให้เข้าใจผิดว่ามีการควบคุม เช่น <code>encrypt</code> หรือ <code>approved_objects</code>	implement การตรวจหรือทำเครื่องหมายว่ายังไม่ใช้
F9	ต่ำ	โค้ดและเอกสารยังไม่ commit	<code>api.py</code> , <code>__main__.py</code> , <code>docs/</code>	production ต่างจาก repo และเอกสารยังไม่อยู่ใน Git	review แล้ว commit และ deploy
F10	ต่ำ	Swagger โหลจาก unpkg	api.py	แก้แล้ว: pin เป็น 5.32.15 ; ยังควรพิจารณา vendor asset/SRI ใน hardening รอบถัดไป	
F11	ต่ำ	SELECT guard เป็นการตรวจข้อความ	sqlserver.py (<code>select_approved</code>)	แก้แล้ว: บังคับคำสั่งเดียว ห้าม <code>;</code> /comment และเพิ่ม CREATE/GRANT/REVOKE/DENY/BACKUP/RESTORE/WAITFOR/BULK/OPENROWSET; ยังต้องพึงสิทธิ์ read-only ของ DB	
R1	สูง	guard เติมยอมให้มีหลาย statement ต่อท้าย SELECT	sqlserver.py	แก้แล้ว: ปฏิเสธ <code>;</code> และ comment พร้อม regression test	
R2	สูง	guard <code>IsApprSo</code> ตรวจแค่ว่ามีค่านี้	sqlserver.py	แก้แล้ว: ต้องมี <code>IsApprSo = 1</code> และห้ามเทียบกับค่าอื่น	
R3	สูง	detail ที่ไม่มี <code>item_code</code> ถูกทิ้งเจียบ	sqlserver.py	แก้แล้ว: ทั้ง SO เป็น <code>review/detail_item_code_missing</code>	
R4	สูง	SO ที่ค้าง <code>needs_review</code> ถูกข้ามเจียบทุกรอบ	service.py	แก้แล้ว: รายงาน <code>needs_review/prior_attempt_*</code> และไม่ส่งซ้ำอัตโนมัติ	
R5	สูง	log อาจพ่นข้อความ driver และ <code>order_no</code> ดิบ	<code>__main__.py</code> , <code>service.py</code>	แก้แล้ว: log เฉพาะ class name, นับ state และ sanitize identifier	
R6	กลาง	isolation ไม่ครอบชั้นอ่าน/map	service.py	แก้แล้ว: batch ที่อ่านไม่ได้คืนผล <code>review</code> แทนการโยน exception	
R7	กลาง	DO callback พร้อมกันได้ HTTP 500	service.py	แก้แล้ว: จอง identity ใน BEGIN IMMEDIATE และมี concurrent test	
R8	กลาง	ERP/eVRP ใช้ <code>receipt_id</code> ซนกัน	service.py, <code>api.py</code>	แก้แล้ว: identity เป็น <code><source>:<receipt_id></code> พร้อม migration	
R9	กลาง	<code>.dockerignore</code> อยู่ผิดตำแหน่ง	<code>.dockerignore</code>	แก้แล้ว: ย้ายไป root และพิสูจน์ด้วย docker build context check	

14.2 จุดแข็งที่พบ

- ใช้ idempotency แบบ transaction (BEGIN IMMEDIATE) มี test การส่งซ้ำพร้อมกัน และไม่ resend อัตโนมัติเมื่อผลไม่แน่นอน
- เทียบ API key แบบ constant-time, จำกัดขนาด body, ปฏิเสธ NaN และปิด access log เพื่อป้องกันข้อมูลรั่ว
- connector บังคับ HTTPS, ไม่ follow redirect และจำกัดขนาด response ที่ 4 MiB
- container รันแบบ non-root บน root filesystem ที่เป็น read-only, ใช้ no-new-privileges และเปิด app เฉพาะใน network ภายใน
- credential ของ SQL แยกจาก connection string และ escape ค่าให้ถูกต้อง

14.3 เอกสารเดิมที่ไม่ตรงกับโค้ด

เอกสาร	ข้อความเดิม	ความจริงตามโค้ด
README.md บรรทัดต้น	SQL connector ยังไม่ได้ implement	มี connector แล้ว แต่รอ query/mapping และ read-only verification
README.md หัวข้อ webhook	SQL sweep worker จะ implement ใน Phase 04	ใช้ scheduled approved-SO worker; Hook ไม่ใช่ช่องทางหลัก
README.md สัญญา API	ทุก endpoint ยกเว้น /health ต้องใช้ key	/docs , /openapi.json และ /favicon.ico ก็เป็นสาธารณะ
Integration_Test_Plan.md ERP-04	event_id เดิมที่ข้อมูลเปลี่ยนจะได้ 409	โค้ดตรวจ payload hash และตอบ conflict
Integration_Test_Plan.md VRP-03	มี controlled retry และ attempt record	ทดสอบว่าตั้ง needs_review และไม่ resend อัตโนมัติ; ยังไม่มี attempt history
Thaisausage_Operations_Guide.md หัวข้อ 3	config/local.json ต้องกำหนด source และ approved query set	โค้ดยังไม่ได้อ่าน key เหล่านี้ (F8)

15. เอกสารที่เกี่ยวข้อง

- docs/ERP_Integration_Guide.md — สำหรับทีม ERP: webhook, JSON contract และ retry
- docs/Thaisausage_Operations_Guide.md — release gates และกติกากาการกู้คืน
- docs/Integration_Test_Plan.md — test case และแบบฟอร์ม sign-off
- docs/erp-source-contract.md — รายการข้อมูลที่ต้องได้จาก ERP/DBA ใน Phase 01
- docs/do-insert-contract.md — mapping matrix ของ tbl_D0hdr / tbl_Dodtl สำหรับ DBA
- process/features/erp-sqlserver/active/PHASE_07_D0_INSERT_PLAN_16-09-26.md — แผน DO Insert
- docs/openapi.json และ https://thaisausage.krs.co.th/docs
- process/features/erp-sqlserver/active/ERP_SQLSERVER_PLAN_12-09-26.md — แผน 6 phase